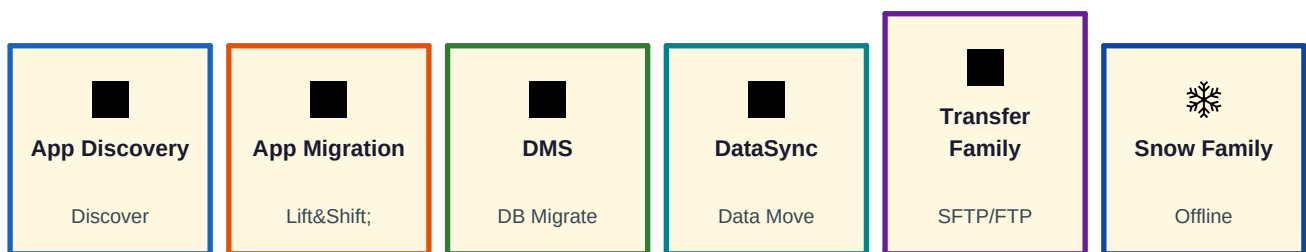


AWS Migration & Transfer Services

Comprehensive Learning Guide

Application Discovery · App Migration (MGN) · DMS · DataSync · Transfer Family · Snow Family

Beginner-Friendly · In-Depth · Practical · Hands-On



AWS Solutions Architect Training Series • Version 1.0 • 2025

Table of Contents

■ AWS Application Discovery Service

- 1. Simple Introduction
- 2. Core Concepts
- 3. Architecture & Working
- 4. Features
- 5. Real-World Use Cases
- 6. Practical Example
- 7. Integrations
- 8. Comparison

■ AWS Application Migration Service (MGN)

- 1. Simple Introduction
- 2. Core Concepts
- 3. Architecture & Working
- 4. Features
- 5. Real-World Use Cases
- 6. Practical Example
- 7. Integrations
- 8. Comparison

■ AWS Database Migration Service (DMS)

- 1. Simple Introduction
- 2. Core Concepts
- 3. Architecture & Working
- 4. Features
- 5. Real-World Use Cases
- 6. Practical Example
- 7. Integrations
- 8. Comparison

■ AWS DataSync

- 1. Simple Introduction
- 2. Core Concepts
- 3. Architecture & Working
- 4. Features
- 5. Real-World Use Cases
- 6. Practical Example
- 7. Integrations
- 8. Comparison

■ AWS Transfer Family

- 1. Simple Introduction
- 2. Core Concepts
- 3. Architecture & Working

- 4. Features
- 5. Real-World Use Cases
- 6. Practical Example
- 7. Integrations
- 8. Comparison

AWS Snow Family

- 1. Simple Introduction
- 2. Core Concepts
- 3. Architecture & Working
- 4. Features
- 5. Real-World Use Cases
- 6. Practical Example
- 7. Integrations
- 8. Comparison

■ AWS Application Discovery Service

Discover On-Premises Infrastructure Before You Migrate

1. Simple Introduction

AWS Application Discovery Service (ADS) helps enterprise customers **plan migration projects by collecting data about their on-premises data centres**. Before you move anything to the cloud, ADS discovers every server, maps its configuration, tracks utilisation, and shows you the dependencies between applications — so you know exactly what you're migrating and in what order.

■ **Real-World Analogy:** Imagine you're planning to move a large warehouse to a new building. Before hiring movers, you need a complete inventory: every shelf, box, and item, plus a map of which items are used together. ADS is that inventory team — it walks through your data centre and catalogues everything so you can plan the move without surprises.

Business Problem It Solves:

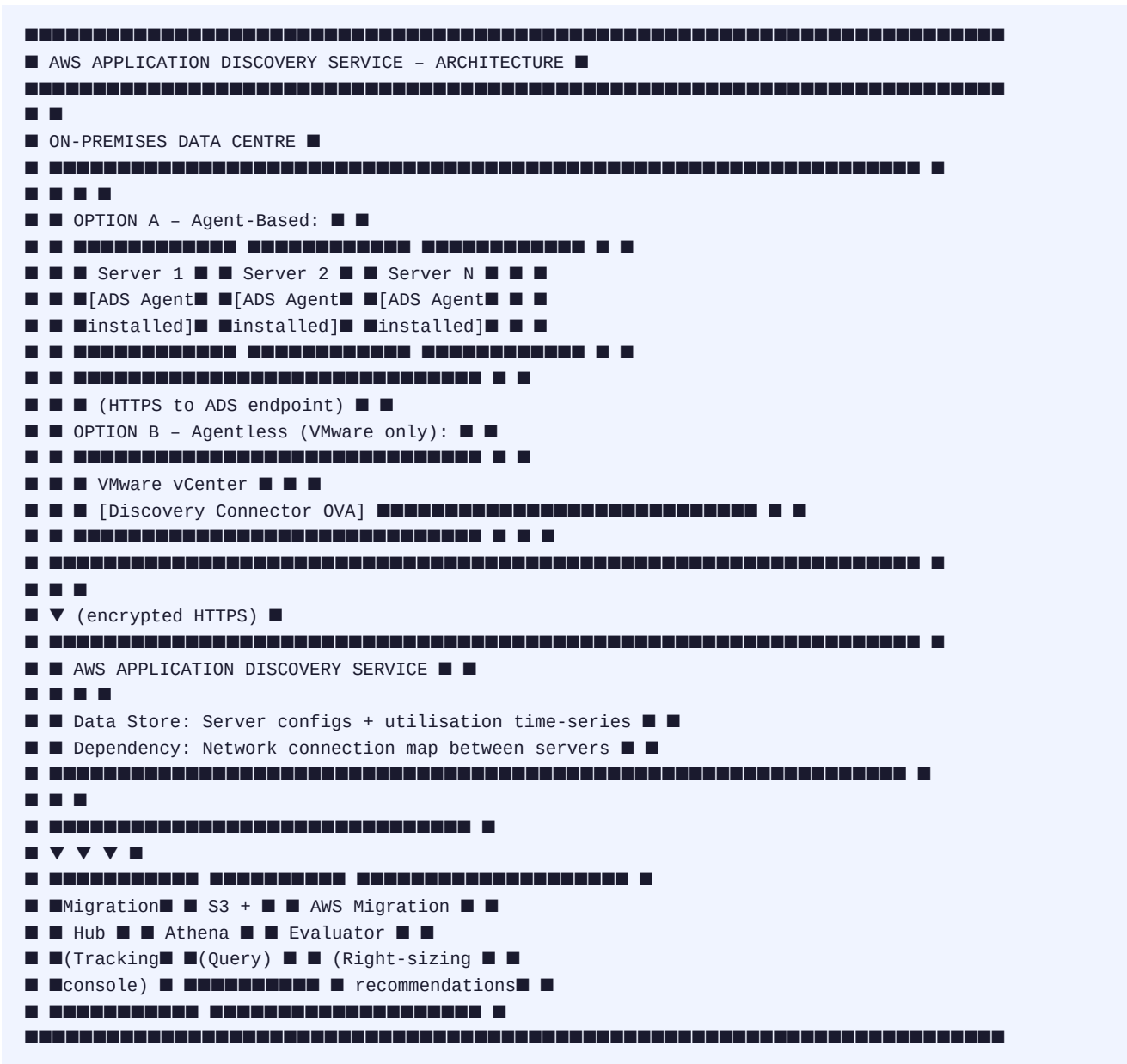
- Most enterprises don't have an accurate, up-to-date inventory of their own servers
- Without dependency mapping, migrating one app can silently break 10 others
- Right-sizing cloud instances is impossible without real utilisation data
- ADS feeds data directly into AWS Migration Hub for centralised tracking

2. Core Concepts

Term	Definition	Example
Agent-Based Discovery	Lightweight AWS agent installed on each server; collects OS metrics, processes, network connections	Installed on 200 Windows/Linux servers in DC
Agentless Discovery	VMware vCenter connector — no agent needed; reads VM metadata from hypervisor	Discovers 500 VMware VMs without touching each one
Discovery Connector	Virtual appliance deployed in VMware vCenter for agentless discovery	OVA deployed in vCenter, connects to ADS
Discovery Agent	Software installed on physical or virtual servers for deep discovery	Collects CPU, RAM, disk, network, running processes
Application	A logical grouping of discovered servers that work together	Three-tier app: web + app + DB servers grouped as 'ERP-App'
Dependency Mapping	Shows which servers talk to which others over the network	Web server → App server:8080 → DB server:5432
Migration Hub	Central console that consumes ADS data to track migration progress	See all 500 servers, their migration status in one place
Data Exploration	Query raw discovery data in Amazon Athena via S3 export	Run SQL to find all servers with > 8 CPU cores

Term	Definition	Example
Tag	Key-value labels applied to discovered servers for organisation	env=production, team=finance, criticality=high
Utilisation Data	Time-series CPU, memory, disk, network data per server	Server X: avg CPU 12%, peak 78%, RAM 6/16 GB used

3. Architecture & Working



Phase	Step	Action
Setup	1	Deploy Discovery Connector (VMware) or install Discovery Agents on servers
Discovery	2	Agents/connector collect server metadata, processes, network connections
Transmit	3	Data sent securely over HTTPS to ADS endpoint in AWS

Phase	Step	Action
Explore	4	View servers in Migration Hub console; group into applications
Analyse	5	Export to S3; query with Athena; feed into Migration Evaluator
Plan	6	Define migration waves based on dependency groups and priorities

4. Features

Feature	Description	Benefit
Agentless VMware Discovery	Discover all VMware VMs via a single connector OVA without touching each server	Discover hundreds of VMs in hours, not weeks
Agent-Based Deep Discovery	Collect OS-level data: running processes, TCP connections, performance metrics	Detailed dependency and performance data for accurate planning
Dependency Mapping	Visualise which servers communicate with which, on which ports	Group servers into migration waves without breaking dependencies
Continuous Utilisation Data	Collect CPU, memory, disk, network metrics over time	Right-size cloud instances — avoid over- or under-provisioning
Migration Hub Integration	Discovered servers flow directly into AWS Migration Hub tracking	One pane of glass for the entire migration programme
S3 Data Export + Athena	Export raw discovery data to S3; query it with Amazon Athena SQL	Custom analysis, BI dashboards, migration reports
Application Grouping	Group related servers into logical application units	Plan migration waves at the application level
Migration Evaluator Integration	Feed utilisation data into Migration Evaluator for TCO analysis	Business case and ROI calculation for migration project
Tag & Filter	Tag servers with custom key-value pairs; filter and search	Organise thousands of servers by team, environment, criticality

5. Real-World Use Cases

Industry	Use Case
Enterprise	Large bank discovers 3,000 servers across 5 DCs; maps 180 application dependencies before planning migration waves
Healthcare	Hospital system discovers which clinical systems share databases to avoid migrating them in wrong order
Retail	E-commerce company identifies 40% of servers are idle (< 5% CPU) — candidates for decommission, saving cloud costs

Industry	Use Case
Manufacturing	ERP discovery reveals undocumented middleware servers connecting SAP to legacy MES systems
M&A;	Acquiring company uses ADS to rapidly inventory the target company's entire IT estate post-acquisition
Government	Agency discovers on-prem infrastructure to produce accurate FISMA compliance inventory before cloud migration

6. Practical Example

Scenario: Discover your VMware environment using the agentless Discovery Connector.

```
# Step 1 - Download the OVA and deploy in vCenter (done via console) # Then configure via AWS CLI:
# Step 2 - Create the Connector (after OVA is deployed and registered) aws discovery
start-data-collection-by-agent-ids \ --agent-ids c12345678-abcd-1234-efgh-123456789012 # Step 3 -
List all discovered servers aws discovery describe-configurations \ --configuration-type SERVER \
--filters 'name=server.osName,values=Linux,condition=CONTAINS' # Step 4 - Start continuous export
to S3 for Athena querying aws discovery start-continuous-export # Step 5 - Create an Application
and add servers to it aws discovery create-application \ --name 'ERP-Production' \ --description
'SAP ERP 3-tier application' aws discovery associate-configuration-items-to-application \
--application-configuration-id a-123456789 \ --configuration-ids s-111 s-222 s-333 # Step 6 -
Describe network connections (dependency map) aws discovery list-configurations \
--configuration-type CONNECTION \ --max-results 50
```

■ **Result:** All VMware VMs are now visible in AWS Migration Hub with CPU/RAM metrics and network dependency maps. Group them into apps and feed directly into AWS Application Migration Service for the actual lift-and-shift.

7. Integrations

AWS Service	Integration
AWS Migration Hub	Primary consumer of ADS data — tracks migration progress of discovered servers
AWS Migration Evaluator	Ingests utilisation data to produce TCO comparisons and right-sizing recommendations
Amazon S3	Continuous export of raw discovery data for long-term analysis
Amazon Athena	Query S3-exported discovery data with SQL for custom reports
Amazon QuickSight	Build dashboards on top of Athena/S3 discovery data
AWS Application Migration Service (MGN)	Discovered servers become migration source in MGN
AWS DMS	Discovered databases feed into DMS migration planning
AWS IAM	Fine-grained access to discovery data and console

8. Comparison

Feature	ADS	Manual Inventory	Third-Party (Flexera/RISC)	AWS Systems Manager Inventory
Discovery method	Agent / Agentless	Manual spreadsheets	Agent/agentless	SSM Agent (AWS only)
Dependency mapping	■ TCP connections	■ Manual	■ Advanced	■ No
Cloud integration	■ Native AWS	■ None	■ Via connectors	■ Native AWS
Agentless VMware	■ Yes	■ No	■ Yes	■ No
Cost	Free (pay for storage)	Staff time	License fee	Free
On-prem servers	■ Physical + virtual	■ Any	■ Any	■ AWS only
Best for	Pre-migration planning	Very small estates	Large complex estates	Post-migration mgmt

■ AWS Application Migration Service (MGN)

Lift-and-Shift Any Application to AWS with Minimal Downtime

1. Simple Introduction

AWS Application Migration Service (MGN) is the **recommended service for lift-and-shift (rehost) migrations** to AWS. It continuously replicates your source servers — physical, virtual, or cloud — to AWS, keeping them in sync. When you're ready to cut over, it launches identical EC2 instances in your target AWS account with minimal downtime — typically minutes.

■ **Real-World Analogy:** MGN is like a **live mirror truck**. While your warehouse is still operating, the truck quietly copies every item to the new warehouse in real time. When you're ready to move, the gap between the two warehouses is tiny (just seconds of changes). You flip the switch, the new warehouse opens, and customers barely notice the transition.

Business Problems Solved:

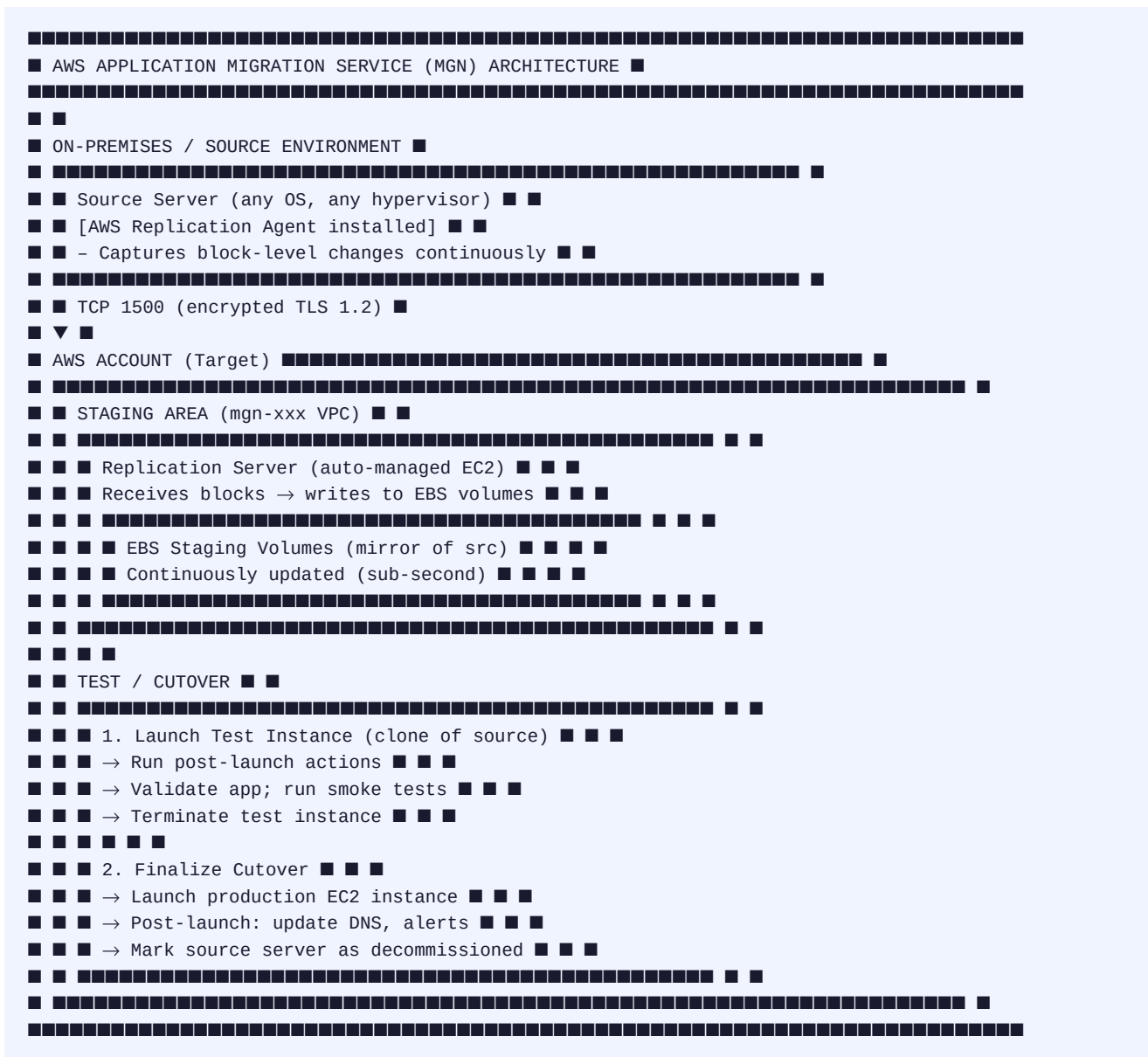
- Migrate any server to EC2 without re-architecting or changing the application
- Minimise downtime — final cutover window is typically under 1 hour
- Automate repetitive migration tasks across hundreds of servers
- Test migrations safely before final cutover without impacting production
- Supports any source: VMware, Hyper-V, bare metal, Azure, GCP, or other clouds

2. Core Concepts

Term	Definition	Example
Source Server	The on-premises or cloud server being migrated	Windows Server 2016 running Oracle EBS
Replication Agent	Lightweight agent installed on source server; streams block-level data to AWS	AWS Replication Agent installed on each server
Replication Server	Temporary EC2 instance in your account that receives block data from agents	c5.large instance created automatically by MGN
Staging Area	The EBS volumes in AWS that mirror source server disks in real time	GP3 EBS volumes matching source server disk sizes
Test Instance	Launch a non-production copy of the migrated server to validate before cutover	Launch test EC2 to verify app works — no production impact
Cutover Instance	The final production EC2 instance created at cutover time	m5.large EC2 replacing the on-prem server permanently
Launch Template	EC2 configuration (instance type, VPC, subnets, SGs) for test/cutover instances	Target instance type, VPC settings, IAM role
RPO	Recovery Point Objective — how much data could be lost; MGN achieves sub-second RPO	Sub-second for all replicated servers

Term	Definition	Example
RTO	Recovery Time Objective — time to cutover; MGN achieves minutes	Typically < 1 hour cutover window
Lifecycle State	Current state of the server in MGN: Not ready / Ready / Test / Cutover	Server moves from Replicating → Ready → Test → Cutover
Post-Launch Actions	Scripts that run automatically after EC2 launch (test or cutover)	Update DNS, install monitoring agent, run smoke tests
Wave	A batch of servers migrated together in a coordinated group	Wave 1: web tier; Wave 2: app tier; Wave 3: DB tier

3. Architecture & Working



Phase	Activity
1. Install Agent	Install AWS Replication Agent on source servers — starts initial sync

Phase	Activity
2. Initial Replication	Full disk copy to EBS staging volumes (can take hours depending on disk size)
3. Continuous Replication	Only changed blocks replicated continuously — sub-second lag
4. Configure Launch Settings	Set target EC2 instance type, VPC, subnets, security groups
5. Launch Test Instance	Spin up a test EC2 from current staging snapshot; validate the migrated app
6. Mark Ready for Cutover	Confirm test passed; schedule cutover window
7. Cutover	Stop source server, replicate final changes, launch production EC2
8. Post-Launch Actions	DNS update, monitoring agent, decommission source server

4. Features

Feature	Description
Any Source Support	Migrate from VMware, Hyper-V, physical servers, AWS, Azure, GCP — any OS
Sub-Second RPO	Block-level continuous replication keeps staging EBS within sub-second of source
Non-Disruptive Testing	Launch test EC2 instances without stopping source or impacting production
Post-Launch SSM Actions	Run automation documents (SSM) after launch — configure, validate, notify
Migration Dashboard	Track status of every server (replicating, ready, testing, cutover) in one view
Replication Templates	Define default instance types, volume types, network settings for all servers
Waves & Applications	Group servers into logical apps and waves for coordinated migration
Data Encryption	Data in transit encrypted with TLS; EBS volumes encrypted with KMS at rest
Free Migration Service	MGN itself is free; you pay only for EC2/EBS replication resources
Bandwidth Throttling	Limit replication bandwidth to avoid impacting production network
Cross-Region Migration	Migrate servers to any AWS region or AWS GovCloud

5. Real-World Use Cases

Industry	Scenario
Retail	Migrate 500 on-prem application servers to AWS in 90 days to exit a data centre lease
Banking	Rehost core banking middleware to EC2 with < 30-minute cutover window during maintenance

Industry	Scenario
Healthcare	Migrate clinical applications to AWS without re-architecting to meet a compliance deadline
Manufacturing	Exit an aging data centre by rehosting all Linux/Windows ERP servers to EC2
Startup	Move from a co-located server to AWS to gain elasticity and reduce CapEx
Cross-Cloud	Migrate Azure/GCP workloads to AWS with the same agent-based approach
Disaster Recovery	Use MGN replicas as standby DR instances — fast RTO if primary site fails

6. Practical Example

Scenario: Migrate a Linux web server to EC2 using MGN CLI.

```
# Step 1 - Initialize MGN in your AWS account (first time only) aws mgn initialize-service # Step 2
- Install the replication agent on your source server (run ON the source server) wget -O
./aws-replication-installer-init.py \
https://aws-application-migration-service-.s3.amazonaws.com/latest/linux/\
aws-replication-installer-init.py sudo python3 aws-replication-installer-init.py \ --region
us-east-1 \ --aws-access-key-id AKIAIOSFODNN7EXAMPLE \ --aws-secret-access-key wJalrXUtnFEMI #
Step 3 - List source servers (once agent is installed and syncing) aws mgn describe-source-servers
--filters 'lifeCycleStates=READY_FOR_TEST' # Step 4 - Update launch template (set target instance
type etc.) aws mgn update-launch-configuration \ --source-server-id s-abcdef1234567890 \
--target-instance-type-right-sizing-method BASIC \ --launch-disposition STARTED # Step 5 - Launch
TEST instance aws mgn start-test \ --source-server-i-ds s-abcdef1234567890 # Step 6 - After
validation, mark cutover ready and cut over aws mgn finalize-cutover \ --source-server-i-ds
s-abcdef1234567890 aws mgn start-cutover \ --source-server-i-ds s-abcdef1234567890
```

■ **Result:** Your Linux server is now running as an EC2 instance in AWS. Downtime during cutover was under 15 minutes. The source server is now decommissioned.

7. Integrations

AWS Service	Integration
AWS Migration Hub	MGN reports server status to Migration Hub for centralised tracking
Amazon EC2	Target compute — MGN creates EC2 instances from replicated EBS volumes
Amazon EBS	Staging area volumes; target root and data volumes
AWS Systems Manager (SSM)	Post-launch automation actions run as SSM documents
AWS KMS	Encrypt staging EBS volumes and target volumes at rest
Amazon CloudWatch	Monitor replication lag, agent health, and cutover events
AWS IAM	Roles for replication agents, launch permissions, post-launch scripts
AWS Application Discovery Service	Feeds discovered server list into MGN as migration candidates
Amazon EventBridge	MGN emits lifecycle events — trigger automation on cutover completion
AWS CloudTrail	Audit log of all MGN API calls for compliance

8. Comparison

Feature	MGN (App Migration Svc)	AWS SMS (legacy)	CloudEndure Migration	VM Import/Export
Status	Current recommended	Deprecated (2022)	Superseded by MGN	Still available
Replication	Continuous block-level	Snapshot-based	Continuous block-level	One-time import
RPO	Sub-second	Hours	Sub-second	Point-in-time
Any source	■ Yes	VMware/Hyper-V only	■ Yes	■ Yes
Test before cut	■ Yes	Limited	■ Yes	■ No
Post-launch auto	■ SSM actions	■ No	Limited	■ No
Cost	Free service	Free (deprecated)	Free (part of MGN)	Free (EC2 charges)
Best for	All server migrations	Legacy VMware migrations	Use MGN instead	One-off single imports

■ AWS Database Migration Service (DMS)

Migrate Databases to AWS Quickly, Securely, and with Minimal Downtime

1. Simple Introduction

AWS Database Migration Service (DMS) is a **managed service that migrates databases to AWS quickly, securely, and with minimal downtime**. DMS supports homogeneous migrations (e.g., Oracle to Oracle on RDS) and heterogeneous migrations (e.g., Oracle to Aurora PostgreSQL). During migration, the source database remains fully operational, minimising downtime for applications that depend on the database.

■ **Real-World Analogy:** DMS is like a **professional moving company that copies your library while it stays open**. Readers can still borrow books (database queries continue). The movers copy every book to the new library (initial load) and then a team watches for any new books or changes (CDC — Change Data Capture) and mirrors them instantly. When the new library is fully in sync, you flip the sign — minimal downtime.

Business Problems Solved:

- Escape expensive Oracle/SQL Server licences by migrating to open-source Aurora/PostgreSQL
- Migrate databases with minimal downtime using continuous Change Data Capture (CDC)
- Modernise legacy on-premises databases to cloud-native managed RDS/Aurora
- Consolidate multiple databases into one; or split one database into many
- Replicate data continuously for analytics, DR, or cross-region reads

2. Core Concepts

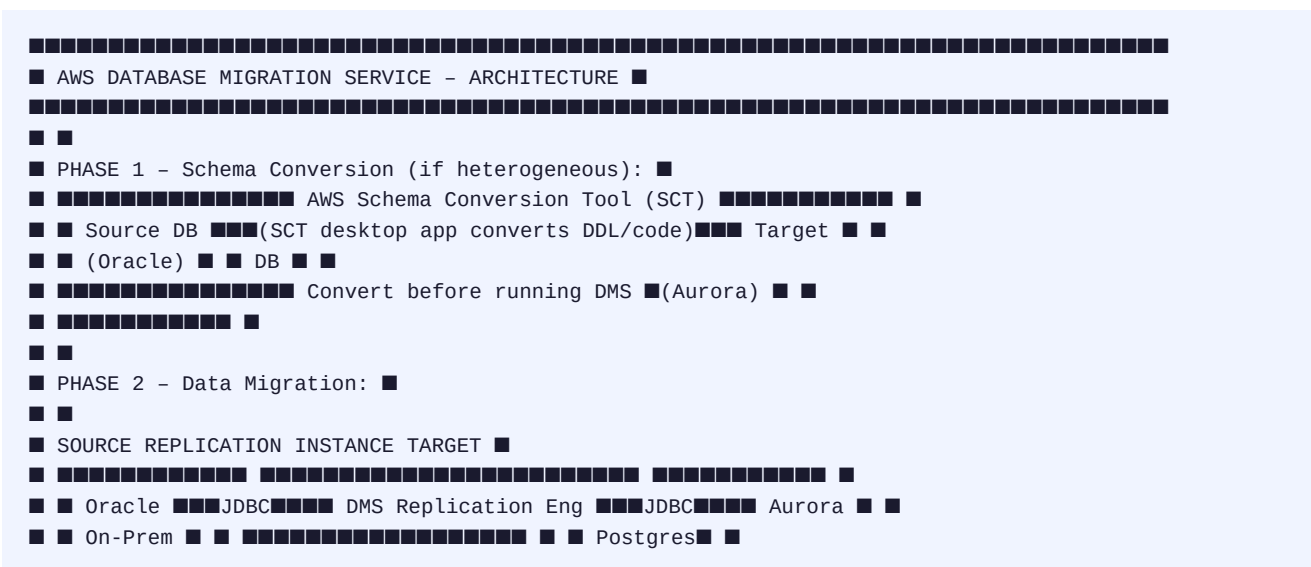
Term	Definition	Example
Replication Instance	A managed EC2 instance that runs the DMS replication engine	dms.r5.xlarge instance handles the migration workload
Source Endpoint	Connection to the source database	Oracle 19c on-premises with host, port, credentials
Target Endpoint	Connection to the destination database	Amazon Aurora PostgreSQL cluster in us-east-1
Replication Task	The actual migration job — defines what to migrate and how	Full Load + CDC task from Oracle to Aurora
Full Load	Copy all existing data from source to target (initial bulk load)	Move 500 GB of historical data in the first pass
CDC	Change Data Capture — continuously capture and apply ongoing changes	After full load, capture every INSERT/UPDATE/DELETE
Full Load + CDC	Most common: bulk load first, then switch to CDC for ongoing sync	Near-zero downtime migration pattern
Schema Conversion	Convert source DDL schema to target DB syntax	Oracle PL/SQL procedures → Aurora PostgreSQL PL/pgSQL

Term	Definition	Example
AWS SCT	AWS Schema Conversion Tool — desktop app to convert schemas before DMS	SCT converts Oracle schema; DMS migrates the data
Table Mapping	Rules defining which tables/columns to include, exclude, or transform	Include only tables in SCHEMA_A; rename column phone → phone_number
Replication Subnet Group	VPC subnets in which DMS creates the replication instance	Private subnets in us-east-1a and us-east-1b
LOB Mode	How DMS handles Large Object columns (BLOBs, CLOBs)	Full LOB mode or Limited LOB mode (truncate at N KB)
Validation	DMS can verify row counts and data checksum between source and target	Confirm 10M rows migrated correctly after full load

Supported Databases

Category	Sources	Targets
Relational	Oracle, SQL Server, MySQL, PostgreSQL, MariaDB, SAP ASE, IBM Db2	RDS (Oracle, SQL Server, MySQL, PostgreSQL, MariaDB), Aurora
NoSQL	MongoDB, DocumentDB (as source)	Amazon DynamoDB, MongoDB, DocumentDB
Data Warehouse	Teradata, Netezza, Greenplum, IBM Informix	Amazon Redshift
Cloud	Azure SQL, GCP Cloud SQL, AWS RDS cross-region	Any AWS-supported target
Streaming	—	Amazon Kinesis Data Streams, Amazon MSK (Kafka)
Object Store	—	Amazon S3 (as a target for CDC audit trail)

3. Architecture & Working



4. Features

5. Real-World Use Cases

Page 16

Industry	Migration Scenario
E-Commerce	On-prem MySQL → Amazon Aurora: gain auto-scaling read replicas for traffic spikes
Healthcare	SQL Server on-prem → RDS SQL Server: move to managed service, eliminate DBA overhead
Analytics	Oracle Data Warehouse → Amazon Redshift: 10x query performance at lower cost
Startup	MongoDB on-prem → Amazon DocumentDB: managed NoSQL with no operational overhead
Retail	Teradata → Redshift: migrate 100 TB data warehouse for cloud-native BI
Global	US-East RDS → EU-West RDS: cross-region ongoing replication for low-latency reads
Gaming	MySQL monolith → DynamoDB: migrate player session table to NoSQL for scale

6. Practical Example

Scenario: Migrate MySQL on-premises to Amazon RDS MySQL using Full Load + CDC.

```
# Step 1 - Create Replication Subnet Group
aws dms create-replication-subnet-group \
--replication-subnet-group-id my-dms-subnet-grp \
--replication-subnet-group-description 'DMS
subnet group' \
--subnet-ids subnet-aaa111 subnet-bbb222 # Step 2 - Create Replication Instance
aws dms create-replication-instance \
--replication-instance-identifier my-dms-instance \
--replication-instance-class dms.r5.xlarge \
--allocated-storage 100 \
--multi-az \
--replication-subnet-group-identifier my-dms-subnet-grp # Step 3 - Create Source Endpoint (on-prem
MySQL)
aws dms create-endpoint \
--endpoint-identifier src-mysql-onprem \
--endpoint-type source \
--engine-name mysql \
--server-name 10.0.1.50 \
--port 3306 \
--username dms_user \
--password 'MySecret123!' # Step 4 - Create Target Endpoint (RDS MySQL)
aws dms create-endpoint \
--endpoint-identifier tgt-rds-mysql \
--endpoint-type target \
--engine-name mysql \
--server-name mydb.cluster-xyz.us-east-1.rds.amazonaws.com \
--port 3306 \
--username admin \
--password 'TargetPass!' # Step 5 - Create and Start the Migration Task (Full Load + CDC)
aws dms
create-replication-task \
--replication-task-identifier mysql-migration-task \
--source-endpoint-arn arn:aws:dms:us-east-1:123:endpoint:src-mysql-onprem \
--target-endpoint-arn
arn:aws:dms:us-east-1:123:endpoint:tgt-rds-mysql \
--replication-instance-arn
arn:aws:dms:us-east-1:123:rep:my-dms-instance \
--migration-type full-load-and-cdc \
--table-mappings '{"rules":[{"rule-type":"selection","rule-id":"1",
"rule-name":"include-all","object-locator":{"schema-name":"%",
"table-name":"%"},"rule-action":"include"}]}'
aws dms start-replication-task \
--replication-task-arn arn:aws:dms:us-east-1:123:task:mysql-migration-task \
--start-replication-task-type start-replication
```

■ **Result:** Full load copies all existing data to RDS. CDC then keeps source and target in sync. When CDC lag reaches 0, stop writes to source, apply final changes, and update the app connection string. Typical downtime: < 5 minutes.

7. Integrations

AWS Service	Integration
Amazon RDS / Aurora	Most common DMS target — migrate to managed relational databases
Amazon Redshift	Migrate OLAP/data warehouse workloads; DMS streams data warehouse tables

AWS Service	Integration
Amazon DynamoDB	Migrate relational tables to DynamoDB (requires table mapping rules)
Amazon S3	Use S3 as a DMS target — write CDC events as CSV/Parquet for data lake ingestion
Amazon Kinesis	Stream CDC events to Kinesis for real-time analytics pipelines
AWS SCT	Schema Conversion Tool converts DDL/code before DMS migrates data
AWS Secrets Manager	Store DB credentials for endpoints — DMS retrieves them securely
Amazon CloudWatch	DMS task metrics (rows, errors, latency) sent to CloudWatch
AWS VPC	Replication instance in private subnet; uses VPC routing to reach source/target
AWS KMS	Encrypt replication storage and data in transit

8. Comparison

Feature	AWS DMS	AWS SCT	pg_dump / mysqldump	Attunity / Qlik
Type	Managed migration service	Schema converter tool	Native DB backup/restore	3rd-party CDC tool
Data move	■ Yes (bulk + CDC)	■ Schema only	■ Yes (bulk only)	■ Yes
Schema conv.	■ No (data only)	■ Yes (across engines)	■ No	Limited
Minimal down	■ CDC-based	N/A	■ Downtime needed	■ CDC-based
Managed	■ Fully managed	Desktop app	■ Manual	■ Self-managed
Heterogeneous	■ Yes (with SCT)	■ Yes	■ Same engine only	■ Yes
Cost	Per instance-hour	Free	Free	License fee
Use when	DB migration with CDC	Cross-engine schema conv.	Simple same-engine copy	Complex CDC needs

■ AWS DataSync

Fast, Automated, Secure Data Transfer Between On-Premises and AWS

1. Simple Introduction

AWS DataSync is a **fully managed data transfer service** that simplifies, automates, and accelerates moving large amounts of data between on-premises storage systems and AWS storage services — or between AWS storage services. DataSync can move data **up to 10x faster** than open-source tools, handles scheduling, monitoring, encryption, and data validation automatically.

■ **Real-World Analogy:** DataSync is like a **high-speed courier service with smart logistics**. Instead of manually driving files from one location to another (rsync over VPN), you hand the package to DataSync. It uses dedicated high-speed lanes (optimised transfer protocol), checks every package is intact (data validation), and can repeat the delivery on a schedule (incremental transfers).

Business Problems Solved:

- Migrate petabytes of NAS/SAN/HDFS data to S3, EFS, or FSx quickly and reliably
- Ongoing sync between on-premises storage and cloud for hybrid workflows
- Archive cold data from on-premises NAS to S3 Glacier automatically
- Transfer data between AWS storage services (S3 to S3, EFS to FSx, etc.)
- Replace slow, error-prone rsync/robocopy scripts with a managed service

2. Core Concepts

Term	Definition	Example
DataSync Agent	VM appliance deployed on-premises to connect to your storage	OVA deployed on VMware to access your NAS
Task	A DataSync job — defines source, destination, and settings	NFS share → S3 bucket, scheduled every night
Location	A defined storage endpoint (source or destination)	nfs://nas.mycompany.com/share OR s3://my-backup-bucket
Task Execution	A single run of a DataSync task	Nightly run at 2 AM transferring 50 GB of new files
Transfer Mode	Changed Files Only (incremental) or All Files (full resync)	Changed Files Only for nightly incremental backup
Bandwidth Limit	Cap the data transfer rate to avoid saturating WAN	Limit to 100 MB/s during business hours
Encryption	Data encrypted in transit (TLS 1.2) and optionally at rest	S3 SSE-S3 or SSE-KMS applied to transferred objects
Data Validation	Verify data integrity with checksums after transfer	Compare SHA-256 checksums of every transferred file

4. Features

Feature	Description
10x Faster Transfers	Custom protocol with parallel streams, larger I/O, and better WAN optimisation vs rsync
Automatic Retry	Failed file transfers automatically retried; no manual re-run needed
Incremental Transfers	Only transfer new or changed files on subsequent runs — saves time and cost
Data Integrity Checks	SHA-256 checksums verified at source and destination after every transfer
Scheduling	Hourly, daily, weekly schedule or on-demand — no cron jobs to manage
Bandwidth Throttling	Set max transfer rate in MB/s by schedule (lower during business hours)
Metadata Preservation	Preserves file timestamps, permissions, ownership, and POSIX attributes
Encryption in Transit	TLS 1.2 always; no plaintext data on the wire
VPC Endpoint Support	Transfer data without touching the public internet using AWS PrivateLink
CloudWatch Integration	Per-task metrics and per-file error logs in CloudWatch
POSIX / Windows ACLs	Preserve file-system permissions when transferring to EFS or FSx
Filtering	Include/exclude files by name pattern, size, age, or last-modified time
Multi-Protocol Support	NFS v3/v4, SMB 2/3, HDFS, S3-compatible object storage

5. Real-World Use Cases

Industry	Use Case
Media & Entertainment	Move 500 TB video archive from on-premises NetApp NAS to S3 Glacier for cold storage
Life Sciences	Nightly sync of genomics research data from on-premises HPC cluster to S3 for analysis
Financial Services	Daily transfer of transaction log files from branch servers to centralised S3 data lake
Healthcare	Migrate PACS (medical imaging) archive from SAN to S3 with HIPAA-compliant encryption
E-Commerce	Migrate product image library (2M files, 4 TB) from NFS NAS to S3 before website relaunch
Backup & DR	Ongoing daily backup of on-premises file servers to S3 as DR copy
Cloud-to-Cloud	Replicate S3 bucket in us-east-1 to EFS in eu-west-1 for GDPR data residency
HPC Migration	Move HDFS data from on-premises Hadoop cluster to S3 before migrating to EMR

6. Practical Example

Scenario: Nightly transfer of new files from an on-premises NFS share to Amazon S3.

```
# Step 1 - Deploy DataSync Agent OVA in your VMware/on-prem environment, # then register the
agent: aws datasync create-agent \ --activation-key AAAAA-BBBBB-CCCCC-DDDDD-EEEE \ --agent-name
onprem-nfs-agent \ --tags Key=env,Value=production # Step 2 - Create the Source Location (NFS
share) aws datasync create-location-nfs \ --server-hostname 10.0.1.20 \ --subdirectory
/exports/data \ --on-prem-config AgentArns=arn:aws:datasync:us-east-1:123:agent:agent-abc # Step 3
- Create the Destination Location (S3 bucket) aws datasync create-location-s3 \ --s3-bucket-arn
arn:aws:s3:::my-datasync-destination \ --s3-storage-class STANDARD \ --s3-config
BucketAccessRoleArn=arn:aws:iam::123:role/DataSyncS3Role # Step 4 - Create the Task with nightly
schedule, changed files only aws datasync create-task \ --source-location-arn
arn:aws:datasync:us-east-1:123:location/loc-src \ --destination-location-arn
arn:aws:datasync:us-east-1:123:location/loc-dst \ --name nightly-nfs-to-s3 \ --schedule
ScheduleExpression='cron(0 2 * * ? *)' \ --options '{ "TransferMode": "CHANGED", "VerifyMode":
"ONLY_FILES_TRANSFERRED", "LogLevel": "TRANSFER", "PreserveDeletedFiles": "REMOVE" }'
```

■ **Result:** Every night at 02:00 AM, DataSync scans the NFS share, transfers only new/changed files to S3, verifies checksums, and logs results to CloudWatch. No rsync scripts, no manual monitoring.

7. Integrations

AWS Service	Integration
Amazon S3	Most common destination — migrate or sync files to any S3 storage class
Amazon EFS	Migrate on-premises NFS to EFS; sync between EFS and S3
Amazon FSx	Migrate to FSx for Windows (SMB), FSx for Lustre, FSx for ONTAP, OpenZFS
AWS Snowcone	Built-in DataSync agent on Snowcone — collect edge data and sync to S3
Amazon CloudWatch	Task execution metrics, per-file transfer logs, error notifications
Amazon EventBridge	DataSync task completion events trigger downstream workflows
AWS CloudTrail	Audit log of all DataSync API calls
AWS KMS	Encrypt data at rest in S3/EFS/FSx with customer-managed keys
AWS Direct Connect	Use dedicated network link for high-throughput, low-latency transfers
AWS PrivateLink	Transfer data without public internet via VPC endpoints

8. Comparison

Feature	DataSync	AWS Transfer Family	S3 Sync (CLI)	Snow Family	Storage Gateway
Transfer speed	10x optimised	Protocol speed	Standard S3 API	Physical	Protocol speed
Protocols	NFS, SMB, HDFS, S3	SFTP, FTPS, FTP, AS2	HTTPS (S3 API)	Physical + NFS/SMB	NFS, SMB, iSCSI
Scheduling	■ Built-in	■ Manual trigger	■ Cron needed	N/A	Continuous

Feature	DataSync	AWS Transfer Family	S3 Sync (CLI)	Snow Family	Storage Gateway
Data validation	■ Checksums	■ No	■ No	■ Yes	■ No
On-prem agent	■ Required	■ No	■ No	Physical device	■ VM appliance
Internet-free	■ VPC endpoints	■ VPC endpoints	■ VPC endpoints	Physical shipping	■ VPC
Best for	Large file migrations	Managed SFTP/FTP server	Simple S3 copies	Offline petabyte xfr	Hybrid file/block/tape

■ AWS Transfer Family

Fully Managed SFTP, FTPS, FTP, and AS2 File Transfers to AWS Storage

1. Simple Introduction

AWS Transfer Family is a **fully managed service for file transfers directly into and out of Amazon S3 or Amazon EFS** using standard protocols: SFTP (SSH File Transfer Protocol), FTPS (FTP over SSL), FTP (File Transfer Protocol), and AS2 (Applicability Statement 2 — used in B2B/EDI workflows). It replaces self-managed SFTP servers with a managed, scalable, highly available service.

■ **Real-World Analogy:** Transfer Family is like a **managed post office box service**. Your trading partners (banks, retailers, suppliers) have always sent files by dropping them in your SFTP mailbox. Instead of running your own SFTP server in a basement, AWS runs a highly available post office for you. Partners drop files in the same way — nothing changes for them — but the files land directly in your S3 bucket.

Business Problems Solved:

- Eliminate self-managed SFTP/FTP servers — no more patching, scaling, or HA configuration
- Provide trading partners a familiar SFTP/FTPS endpoint that stores files directly in S3
- Meet compliance requirements (PCI-DSS, HIPAA) for secure file transfer
- Enable B2B EDI (Electronic Data Interchange) with AS2 protocol support
- Integrate file transfers into event-driven workflows via Lambda and EventBridge

2. Core Concepts

Term	Definition	Example
Server	A Transfer Family endpoint (SFTP/FTPS/FTP/AS2) in your AWS account	sftp://s-abc123.server.transfer.us-east-1.amazonaws.com
Protocol	The file transfer protocol the server supports	SFTP (port 22), FTPS (port 990), FTP (port 21), AS2 (port 443)
User	An SFTP/FTPS/FTP identity that can connect and transfer files	partner-user with /invoices as their home directory
Identity Provider	How users authenticate — Service-Managed (SSH keys) or Custom (Lambda/IdP)	AWS Managed: upload SSH public key per user
Home Directory	The S3 prefix or EFS path that maps to the user's root /	s3://my-bucket/partner-a/ maps to / for partner-a
Logical Directory	Map multiple S3 paths to different directories for one user	/ → s3://inbound, /archive → s3://archive-bucket
AS2	Applicability Statement 2 — EDI protocol used in supply chain B2B	Retailer sends purchase orders to supplier over AS2
AS2 Connector	Send AS2 messages to external partners from your AWS account	Push invoices to partner's AS2 server

[illegible]

4. Features

Feature	Description
Service-Managed Users	Create users with SSH public key auth stored in Transfer Family — no external IdP needed
Custom Identity Provider	Integrate with Microsoft AD, Okta, or any IdP via Lambda for username/password auth
Managed Workflows	Chain post-upload steps: copy, decrypt, custom Lambda, tag — fully managed pipeline
Logical Directories	Map different S3 buckets/prefixes to a user's virtual directory tree
AS2 Connector	Send AS2 messages outbound to external partners — full two-way AS2 support
VPC Endpoint	Create internet-facing or internal VPC endpoint — private transfers over Direct Connect
Elastic IP Support	Assign static IPs to internet-facing server for allowlisting at partner firewalls
CloudWatch Metrics	Monitor connections, bytes transferred, errors per server and per user
Lambda Auth	Custom Lambda function handles authentication — integrate any identity source
Multi-Protocol Server	One server can support SFTP + FTPS + FTP simultaneously
HA + Global	Multi-AZ by default; deploy in any of 20+ AWS regions
Compliance	FIPS 140-2 validated endpoints available for government/regulated workloads

5. Real-World Use Cases

Industry	Use Case
Retail / Supply Chain	Receive purchase orders and send invoices to 200 suppliers via AS2 EDI protocol
Banking	SFTP server for clients to upload payment files (NACHA, SEPA) — replaces internal SFTP infra
Healthcare	Hospital receives HL7 / FHIR files from 50 clinics via SFTP — files land in S3 for processing

Industry	Use Case
Insurance	Claims files from brokers arrive via SFTP; Managed Workflow decrypts PGP, validates, routes
Government	FTPS server for citizens to submit tax returns — FIPS 140-2 compliance requirement
Media	Advertising agencies deliver video/audio assets via SFTP directly to S3 for editing pipeline
Logistics	Freight forwarders exchange EDI 850/856/810 documents over AS2 with trading partners
Payroll	HR software sends payroll files to bank via SFTP; Transfer Family replaces the bank's legacy server

6. Practical Example

Scenario: Create a managed SFTP server where trading partners upload files directly to S3, with a post-upload workflow to copy to a processing folder.

```
# Step 1 - Create the SFTP Server aws transfer create-server \ --protocols SFTP \
--identity-provider-type SERVICE_MANAGED \ --endpoint-type PUBLIC \ --logging-role
arn:aws:iam::123:role/TransferLoggingRole \ --tags Key=Name,Value=partner-sftp-server # Step 2 -
Create an SFTP User (partner-a) aws transfer create-user \ --server-id s-abc1234567890abcd \
--user-name partner-a \ --role arn:aws:iam::123:role/TransferS3Role \ --home-directory-type
LOGICAL \ --home-directory-mappings \ '["Entry":"/", "Target":"/my-transfer-bucket/partner-a/"]]'
# Step 3 - Add SSH Public Key for the user aws transfer import-ssh-public-key \ --server-id
s-abc1234567890abcd \ --user-name partner-a \ --ssh-public-key-body 'ssh-rsa AAAAB3NzaC1yc2E...
partner-a@company.com' # Step 4 - Create a Managed Workflow (copy + tag on upload) aws transfer
create-workflow \ --description 'Post-upload processing' \ --steps '[
{"Type": "COPY", "CopyStepDetails": {"Name": "copy-to-processed",
"DestinationFileLocation": {"S3FileLocation": {"Bucket": "my-processed-bucket",
"Key": "${Transfer:UserName}/${Transfer:UploadDate}/"} }},
{"Type": "TAG", "TagStepDetails": {"Name": "tag-processed",
"Tags": [{"Key": "status", "Value": "uploaded"}, {"Key": "partner", "Value": "${Transfer:UserName}"} ]}]'
```

■ **Result:** partner-a can now SFTP to the server endpoint and drop files into their virtual / directory. Files land in s3://my-transfer-bucket/partner-a/ and are automatically copied to the processing bucket with tags applied.

7. Integrations

AWS Service	Integration
Amazon S3	Primary storage — all transferred files land directly in S3 buckets
Amazon EFS	Alternative storage — files stored on NFS file system accessible from EC2
AWS Lambda	Custom identity provider auth; Managed Workflow custom steps
Amazon EventBridge	Transfer Family emits events on file upload — trigger downstream automation
Amazon CloudWatch	Server metrics (connections, bytes, errors) and access logs
AWS Directory Service	Authenticate users via Microsoft Active Directory
AWS Secrets Manager	Store FTP/FTPS user passwords securely; Lambda retrieves during auth

AWS Service	Integration
AWS KMS	Encrypt files at rest in S3/EFS with customer-managed keys
AWS CloudTrail	Audit all Transfer Family API calls
Amazon Route 53	Custom hostname (sftp.mycompany.com) via CNAME to Transfer endpoint

8. Comparison

Feature	Transfer Family	Self-Managed SFTP (EC2)	DataSync	S3 Pre-signed URLs
Protocol	SFTP/FTPS/FTP/AS2	Any (SSH/FTP)	NFS/SMB/HDFS/S3	HTTPS only
Managed	■ Fully managed	■ You manage	■ Fully managed	■ Serverless
High Avail.	■ Multi-AZ auto	Manual (ASG + EFS)	■ Yes	■ Yes (S3)
User auth	SSH keys / AD / IdP	OS-level, PAM, LDAP	N/A (IAM)	Pre-signed token
Audit logs	■ CloudWatch	Manual syslog setup	■ CloudWatch	■ S3 access logs
Patching	AWS handles	You patch the server	AWS handles	N/A
B2B / EDI	■ AS2 native	Possible (manual)	■ Not designed	■ No
Best for	Partner file exchange	Full OS-level control	Large data migrations	Simple one-off downloads

AWS Snow Family

Move Petabytes of Data Physically When the Network Is Not Enough

1. Simple Introduction

The AWS Snow Family is a collection of **physical devices that help you migrate large amounts of data into and out of AWS, and run compute at the edge** — in locations without reliable internet connectivity. When moving terabytes or petabytes over the internet would take weeks or months, AWS ships you a rugged device, you load your data, ship it back, and AWS ingests it into S3 — often faster, cheaper, and more reliable than network transfer.

■ **Real-World Analogy:** The Snow Family is like **Amazon's own armoured Brinks truck**. When you need to move a vault of gold bars (petabytes of data), it's faster and safer to put them in the truck and drive them over than to wire-transfer through the banking system (slow internet). AWS ships you the truck, you fill it with data, and they drive it back to their data centre and load it into S3.

Business Problems Solved:

- Transferring petabytes over a 1 Gbps link would take 74+ days — Snow does it in 1-2 weeks
- Run compute and ML inference at edge sites with no internet (ships, oilrigs, military)
- Collect data at remote sites (Antarctic research stations, disaster zones) for cloud upload
- Secure, tamper-evident data transfer with 256-bit encryption and trusted platform module
- Data centre decommission — move entire DC to AWS in one bulk offline transfer

2. Core Concepts

Feature	Snowcone	Snowball Edge Storage	Snowball Edge Compute	Snowmobile
Storage	8 TB HDD / 14 TB SSD	80 TB usable	28 TB NVMe + 32 TB	Up to 100 PB
Compute	2 vCPU, 4 GB RAM	40 vCPUs, 80 GB RAM	52 vCPUs, 208 GB RAM	None
GPU	■ No	■ No	■ Optional NVIDIA	■ No
Weight	4.5 lbs (2 kg)	49.7 lbs (22.5 kg)	49.7 lbs (22.5 kg)	45-foot truck
DataSync Agent	■ Built-in	■ Yes	■ Yes	N/A
IoT Greengrass	■ Yes	■ Yes	■ Yes	■ No
Lambda on device	■ Yes	■ Yes	■ Yes	■ No
EC2 on device	■ No	■ Sbe2/Sbe-c	■ Sbe-c/g	■ No
Clustering	■ No	Up to 15 nodes	Up to 15 nodes	N/A
Use case	Edge collect, small migration	Bulk data migration	Edge ML, video analytics	Data centre migration

[illegible]

4. Features

Feature	Description
256-bit Encryption	All data encrypted on device using KMS — encrypted before it leaves your hands
Tamper-Evident	Tamper-evident enclosure + TPM chip — detects physical tampering; device unusable if tampered
NIST 800-88 Erasure	AWS erases all data after ingestion using NIST-certified secure erase
S3-Compatible API	Your existing S3 code (aws s3 cp, boto3) works with Snow devices unchanged
NFS / SMB Mount	Mount device as a network share — no code changes for legacy apps
Edge Compute (Lambda)	Run Lambda functions locally on the device for pre-processing before upload
Edge Compute (EC2)	Run EC2-compatible instances on Snowball Edge Compute for full workloads
AWS IoT Greengrass	Deploy IoT/ML models at the edge; device acts as a Greengrass core
Cluster Mode	Up to 15 Snowball Edge devices clustered for 1.2 PB capacity and resilience
OpsHub GUI	Point-and-click management — no AWS CLI knowledge needed on-site
DataSync Agent Built-in	Snowcone has a DataSync agent pre-installed for automated online sync
Chain of Custody Tracking	Track device location from creation through delivery to data ingestion

5. Real-World Use Cases

Industry	Device	Use Case
Data Centre Migration	Snowmobile	Move 80 PB from decommissioning DC to S3 in weeks — would take 20+ years over 1 Gbps

Industry	Device	Use Case
Oil & Gas	Snowball Edge Compute	Seismic data collection on rig; run ML models locally; ship processed results back
Military / Defense	Snowball Edge	Collect intel data in disconnected field environments; import to cloud when back at base
Media Production	Snowball Edge Storage	Transfer 40 TB of raw 8K film footage from remote filming location to S3 post-production
Healthcare	Snowball Edge	Archive 15 TB PACS imaging data from rural hospital with 10 Mbps internet to S3
Retail	Snowball Edge	Collect point-of-sale data from stores with no internet; batch-import nightly
Scientific Research	Snowcone	Antarctic research station collects sensor data; ships Snowcone back monthly for S3 upload
Manufacturing	Snowball Edge Compute	Run quality control ML inference at factory floor; no cloud connectivity needed
Disaster Recovery	Snowball Edge	Rapid onsite recovery — pre-loaded Snowball deployed to disaster site as compute+storage

6. Practical Example

Scenario: Create a Snowball Edge import job to migrate 60 TB from your on-premises NAS to Amazon S3.

```
# Step 1 - Create an Import Job
aws snowball create-job \
  --job-type IMPORT \
  --resources '{"S3Resources":[{"BucketArn":"arn:aws:s3::my-migration-bucket", "KeyRange":{}}]}' \
  --description '60TB NAS Migration' \
  --address-id ADID1234567890ABCDEFG \
  --kms-key-arn arn:aws:kms:us-east-1:123456789:key/mrk-abc123 \
  --role-arn arn:aws:iam::123456789:role/SnowballImportRole \
  --snowball-type EDGE_STORAGE_OPTIMIZED \
  --shipping-option NEXT_DAY \
  --notification '{"SnsTopicARN":"arn:aws:sns:us-east-1:123:snowball-alerts", "JobStatesToNotify":["Complete","Failed"],"NotifyAll":false}'
# Step 2 - Check job status (device ships when state = InTransitToCustomer)
aws snowball describe-job --job-id JID1234567890ABCDEF
# Step 3 - On receipt, use the AWS OpsHub desktop app or CLI to copy data:
# Unlock device via OpsHub using credentials from AWS console
# Then use the S3-compatible endpoint on the device:
aws s3 cp /mnt/nas-data/ s3://my-migration-bucket/ \
  --recursive \
  --endpoint-url https://192.168.1.100:8443 \
  --profile snowball
# Step 4 - Ship device back to AWS
# OpsHub displays the prepaid shipping label
# Call carrier to schedule pickup
# Step 5 - Monitor ingestion status in AWS Console
aws snowball describe-job --job-id JID1234567890ABCDEF
# State will progress: InTransitToAWS → AtAWS → InProgress → Complete
```

■ **Result:** 60 TB migrated in approximately 1 week total (shipping + ingestion). Attempting the same over a 100 Mbps link would take ~55 days. Device securely erased after AWS confirms ingestion complete.

7. Integrations

AWS Service	Integration
Amazon S3	Primary destination for all Snow Family import jobs; also source for export jobs

AWS Service	Integration
AWS DataSync	Built-in DataSync agent on Snowcone for automated online sync after initial offline load
AWS Lambda	Run Lambda functions on Snowball Edge / Snowcone for local edge compute
Amazon EC2	Run EC2-compatible AMIs on Snowball Edge Compute for full workloads at the edge
AWS IoT Greengrass	Deploy IoT and ML models at edge — Snowball acts as a Greengrass core
AWS KMS	Customer-managed KMS key encrypts/decrypts all data on device
Amazon SNS	Job lifecycle notifications (device shipped, ingestion complete) sent via SNS
AWS OpsHub	Desktop GUI to unlock, manage, and monitor Snow devices without CLI
Amazon SageMaker	Deploy trained SageMaker models to Snowball Edge for offline inference
AWS IAM	Role grants Snow service access to S3 bucket for ingestion

8. Comparison

Feature	Snowcone	Snowball Edge Storage	Snowball Edge Compute	Snowmobile	DataSync (online)
Data capacity	8-14 TB	80 TB	28-60 TB	100 PB	Unlimited
Transfer method	Physical	Physical	Physical	Physical truck	Network
Edge compute	■ Limited	■ Moderate	■ Full EC2/GPU	■ No	■ No
Internet needed	■ No	■ No	■ No	■ No	■ Yes
Setup time	Days	Days	Days	Weeks	Hours
Best for	< 14 TB edge collect	10–80 TB migration	Edge ML + compute	DC migration >10 PB	Online incremental sync

Quick-Reference Summary

Service	Type	Key Strength	Best For
■ App Discovery Svc	Discovery & Planning	Dependency mapping + utilisation data	Pre-migration inventory of on-prem servers
■ App Migration (MGN)	Lift-and-Shift	Sub-second RPO, minimal downtime cutover	Rehost any server to EC2 quickly
■ DMS	DB Migration	CDC = near-zero downtime DB migration	Migrate/modernise any database to AWS
■ DataSync	File Data Transfer	10x faster + checksums + scheduling	Large file/object data migration to AWS storage
■ Transfer Family	Managed SFTP/FTP/AS2	No SFTP server to manage; files land in S3	Partner file exchange, B2B EDI
❄️ Snow Family	Physical Transfer	Petabyte-scale offline migration	Slow internet, remote/edge sites, massive data

Migration Decision Flow

START: What are you migrating?

■

Abstract

□ □ □ □

▼ ▼ ▼ ▼

SERVICES

(OS + apps) (SQL/NoSQL) (NAS/SAN) (SFTP/AS2)

11111

▼ ▼ ▼ ▼

Step 1: ADS AWS DMS How much data? Transfer Family

(discover, (Full Load + ■ (SFTP/FTPS/AS2)

map deps) CDC for < 10 TB online?

■ min downtime) ■YES: DataSync

Step 2: MGN ■ ■NO:

```
(replicate + SCT for Is internet fast enough?
```

cutover) schema conv ■YES: DataSync + Direct Connect

```
if cross-engine ■NO: Snow Family (Snowball/mobile)
```